

Assignment Code: DA-AG-018

Anomaly Detection & Time Series | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is Anomaly Detection? Explain its types (point, contextual, and collective anomalies) with examples.

Answer:

Anomaly Detection is the process of identifying observations that differ significantly from the normal pattern of data.

1. Point anomaly: A single unusual observation. Example: a sudden ₹50,000 card transaction.
2. Contextual anomaly: An observation is unusual in a particular context. Example: high electricity use at midnight.
3. Collective anomaly: A group of observations is unusual together. Example: a sequence of failed login attempts.

Question 2: Compare Isolation Forest, DBSCAN, and Local Outlier Factor in terms of their approach and suitable use cases.

Answer:

Isolation Forest: Isolates unusual points by randomly splitting features. It is fast and suitable for large, high-dimensional datasets.

DBSCAN: Groups dense points and labels low-density points as noise. It works well for spatial data and irregular clusters.

LOF: Compares the local density of a point with its neighbours. It is useful when anomalies occur in different local densities.

Question 3: What are the key components of a Time Series? Explain each with one example.

Answer:

A time series mainly contains four components:

1. Trend ? long-term upward or downward movement. Example: yearly growth in online sales.
2. Seasonality ? a repeating pattern at fixed intervals. Example: higher retail sales every December.
3. Cyclical variation ? long-term rises and falls related to economic or business cycles.
4. Residual/Irregular component ? random variation left after removing trend and seasonality. Example: an unexpected demand spike.

Question 4: Define Stationary in time series. How can you test and transform a non-stationary series into a stationary one?

Answer:

A stationary time series has a constant mean and variance over time, and its statistical properties do not depend on time.

Testing: use a visual plot, ACF plot, or the Augmented Dickey-Fuller (ADF) test. A small ADF p-value (typically < 0.05) supports stationarity.

Transformation: apply differencing to remove trend; seasonal differencing can remove seasonality. Log or other transformations can stabilize changing variance.

Question 5: Differentiate between AR, MA, ARIMA, SARIMA, and SARIMAX models in terms of structure and application.

Answer:

AR: Uses past values of the series to predict the current value. Useful for autocorrelated data.

MA: Uses past forecast errors. Useful when shocks/errors have a time-dependent effect.

ARIMA: Combines AR, differencing (I), and MA for non-stationary series.

SARIMA: Extends ARIMA with seasonal AR, differencing, and MA terms for repeating seasonal patterns.

SARIMAX: SARIMA plus external/exogenous variables, useful when factors such as weather or promotions affect the series.

Dataset:

- [NYC Taxi Fare Data](#)
- [AirPassengers Dataset](#)

Question 6: Load a time series dataset (e.g., AirPassengers), plot the original series, and decompose it into trend, seasonality, and residual components

(Include your Python code and output in the code box below.)

Answer:

```
Code:
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
url="https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df=pd.read_csv(url, parse_dates=["Month"], index_col="Month")
result=seasonal_decompose(df["Passengers"], model="multiplicative", period=12)
result.plot(); plt.show()
```

Output: Original series and its Trend, Seasonal and Residual components are displayed.

Question 7: Apply Isolation Forest on a numerical dataset (e.g., NYC Taxi Fare) to detect anomalies. Visualize the anomalies on a 2D scatter plot.

(Include your Python code and output in the code box below.)

Answer:

```
Code:
from sklearn.datasets import load_iris
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
X=load_iris().data[:, :2]
model=IsolationForest(contamination=0.05, random_state=42)
labels=model.fit_predict(X)
plt.scatter(X[:,0],X[:,1],c=labels); plt.xlabel("Feature 1"); plt.ylabel("Feature 2"); plt.show()
print("Anomalies:", (labels==-1).sum())
```

Output: A 2D scatter plot highlights the observations detected as anomalies.

Question 8: Train a SARIMA model on the monthly airline passengers dataset. Forecast the next 12 months and visualize the results.

(Include your Python code and output in the code box below.)

Answer:

```
Code:
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
url="https://raw.githubusercontent.com/brownlee/Datasets/master/airline-passengers.csv"
df=pd.read_csv(url,parse_dates=["Month"],index_col="Month")
model=SARIMAX(df["Passengers"],order=(1,1,1),seasonal_order=(1,1,1,12))
fit=model.fit(dispatch=False)
forecast=fit.forecast(12)
plt.plot(df["Passengers"],label="Actual"); plt.plot(forecast,label="Forecast"); plt.legend(); plt.show()
print(forecast.round(1))
```

Output: The next 12 monthly passenger values are forecast and plotted.

Question 9: Apply Local Outlier Factor (LOF) on any numerical dataset to detect anomalies and visualize them using matplotlib.

(Include your Python code and output in the code box below.)

Answer:

```
Code:
from sklearn.datasets import load_iris
from sklearn.neighbors import LocalOutlierFactor
import matplotlib.pyplot as plt
X=load_iris().data[:, :2]
lof=LocalOutlierFactor(n_neighbors=20, contamination=0.05)
labels=lof.fit_predict(X)
plt.scatter(X[:,0],X[:,1],c=labels); plt.xlabel("Feature 1"); plt.ylabel("Feature 2"); plt.show()
print("Anomalies:", (labels==-1).sum())
```

Output: A 2D matplotlib scatter plot shows the points classified as LOF anomalies.

Question 10: You are working as a data scientist for a power grid monitoring company. Your goal is to forecast energy demand and also detect abnormal spikes or drops in real-time consumption data collected every 15 minutes. The dataset includes features like timestamp, region, weather conditions, and energy usage.

Explain your real-time data science workflow:

- How would you detect anomalies in this streaming data (Isolation Forest / LOF / DBSCAN)?
- Which time series model would you use for short-term forecasting (ARIMA / SARIMA / SARIMAX)?
- How would you validate and monitor the performance over time?
- How would this solution help business decisions or operations?

(Include your Python code and output in the code box below.)

Answer:

1. Preprocess the stream: sort by timestamp, handle missing numeric values with median/imputation, fill categorical values with the mode, and encode region/weather using one-hot encoding.
2. Anomaly detection: use Isolation Forest for fast streaming detection; retrain/update it periodically. LOF can be used when local density is important, while DBSCAN is better for stable cluster-shaped data.
3. Forecasting: use SARIMAX because demand has seasonality and external variables such as weather can improve forecasts.
4. Validation: use time-based train/test splits and rolling validation. Monitor MAE/RMSE for forecasts and precision, recall and F1-score for anomalies. Track drift and retrain when performance falls.
5. Business value: early warnings can identify faults or unusual consumption, improve load planning, reduce operating costs, support preventive maintenance, and improve grid reliability.